

Tarea Corta 1 – Pilas

Esta tarea consiste en la programación, modificación y utilización de las siguientes estructuras de datos:

1. Pila con arreglos (ArrayStack)
2. Pila enlazada (LinkedStack)

Parte 1

Implementar las clases **ArrayStack** y **LinkedStack**. Estas clases deben derivar de la clase abstracta **Stack** vista en clase. A continuación, se detallan los métodos que deben implementarse en cada una de ellas. Se subrayan los métodos que son diferentes a lo implementado en clase.

1. Constructor y destructor

2. void push(E element)

En el caso del ArrayStack, si la pila ya alcanzó su tamaño máximo, debe crearse un nuevo arreglo del doble de tamaño y traspasarse todos los elementos al nuevo arreglo que sustituya al anterior con el manejo de memoria adecuado. Esto debe implementarse de la siguiente manera:

1. Crear un arreglo del doble del tamaño y apuntarlo con un puntero temporal.
2. Copiar los elementos del viejo arreglo al nuevo.
3. Eliminar el arreglo viejo de la memoria.
4. Hacer que el atributo elements apunte al nuevo arreglo.
5. Hacer que el atributo max valga el doble.

3. E pop()

4. E topValue()

5. void clear()

6. bool isEmpty()

7. int getSize()

8. void print()

Parte 2

Esta parte consiste en elaborar un programa que lea expresiones en notación infija básicas y las evalúe mostrando el resultado. Los operadores aceptados son suma (+), resta (-), multiplicación (*), división (/) y exponente (^). También se aceptan los paréntesis para modificar la precedencia de los operadores y números enteros como valores.

Precedencia:

1. Exponente (^)
2. Multiplicación y división (*, /)
3. Suma y resta (+, -)

Primeramente, el programa pregunta al usuario el tipo de pila que desea usar: **ArrayStack** o **LinkedStack**. En el caso de los **ArrayStack**, estos deben ser creados sólo 5 espacios para probar que lo realizado en la parte 1 funciona.

Se parte de un *string* con una expresión leída de la consola. De esta expresión se obtendrán tokens (símbolos) que pueden ser de uno o más caracteres que corresponden a números, operadores o paréntesis. Por ejemplo, en la expresión "23 * (4 / 2)", los tokens serían: "23", "*", "(", "4", "/", "2", ").". Se hace la aclaración porque los números pueden estar conformados por varios caracteres.

Para realizar la evaluación se utilizará el siguiente algoritmo que hace uso de dos pilas: una para almacenar números (*double*) y otra para almacenar operadores y paréntesis (*char*). Si durante la ejecución se tratara de eliminar un elemento de una pila vacía (*stack underflow*), entonces la expresión no está bien formada.

Algoritmo para **evaluar** una expresión infija:

1. Leer toda la expresión en un *string*.
2. Por cada token en la expresión hacer lo siguiente:
 - 2.1. Extraer el token y borrarlo de la expresión
 - 2.2. Si es un número (n)
 - 2.2.1. **Push** n en la pila de números
 - 2.3. Si es operador (op)
 - 2.3.1. Mientras se cumpla que (1) la pila de operadores tenga elementos, (2) que el elemento en el tope de esta pila no es paréntesis izquierdo "(" y (3) que la precedencia del operador en el tope de la pila sea mayor o igual a la precedencia de op.
 - 2.3.1.1. Procesar operación
 - 2.3.2. **Push** op en la pila de operadores.
 - 2.4. Si es paréntesis izquierdo "("
 - 2.4.1. **Push** "(" en la pila de operadores
 - 2.5. Si es paréntesis derecho ")"
 - 2.5.1. Mientras se cumpla que el elemento en el top de la pila de operadores sea diferente de "("
 - 2.5.1.1. Procesar operación
 - 2.5.2. **Pop** de la pila de operadores
3. Mientras se cumpla que la pila de operadores tenga elementos
 - 3.1. Procesar operación
4. Si la expresión está bien formada, la pila de números contendrá como único elemento el resultado de la evaluación.

Algoritmo para **procesar una operación**:

1. B = **Pop** de la pila de números
2. A = **Pop** de la pila de números
3. op = **Pop** de la pila de operadores
4. R = resultado de la operación A op B (según sea el operador)
5. **Push** R en la pila de números

Ejemplos de ejecución

A continuación, se muestran ejemplos de ejecución paso a paso. No es necesario que el programa entregado haga las impresiones de la misma forma que estos ejemplos, la única que se necesita es del resultado final, pero se muestran para aclarar el proceso de evaluación.

Ejemplo 1 – Expresión sencilla sin paréntesis

Escriba la expresión a analizar: 1 + 2

Pila números: []

Pila operadores: []

Expresión: 1+2

Token actual: 1

Es número. Push en pila de números.

Pila números: [1]

Pila operadores: []

Expresión: +2

Token actual: +

Es operador.

Push en la pila de operadores.

Pila números: [1]

Pila operadores: [+]

Expresión: 2

Token actual: 2

Es número. Push en pila de números.

Fin de la expresión, procesando lo que queda en las pilas.

Pila números: [1 2]

Pila operadores: [+]

Expresión:

Procesando operación 1+2=3

Pila números: [3]

Pila operadores: []

Expresión:

El resultado de la evaluación es: 3

Ejemplo 2 – Expresión con operadores de diferente precedencia sin paréntesis

Escriba la expresión a analizar: $5 * 2 / 4 + 3^2 / 3$

Pila números: []
Pila operadores: []
Expresión: $5*2/4+3^2/3$

Token actual: 5
Es número. Push en pila de números.
Pila números: [5]
Pila operadores: []
Expresión: $*2/4+3^2/3$

Token actual: *
Es operador.
Push en la pila de operadores.
Pila números: [5]
Pila operadores: [*]
Expresión: $2/4+3^2/3$

Token actual: 2
Es número. Push en pila de números.
Pila números: [5 2]
Pila operadores: [*]
Expresión: $/4+3^2/3$

Token actual: /
Es operador.
Operador * tiene precedencia mayor o igual que /.
Procesando operación $5*2=10$

Push en la pila de operadores.
Pila números: [10]
Pila operadores: [/]
Expresión: $4+3^2/3$

Token actual: 4
Es número. Push en pila de números.
Pila números: [10 4]
Pila operadores: [/]
Expresión: $+3^2/3$

Token actual: +
Es operador.
Operador / tiene precedencia mayor o igual que +.
Procesando operación $10/4=2.5$

Push en la pila de operadores.
Pila números: [2.5]
Pila operadores: [+]
Expresión: $3^2/3$

Token actual: 3
Es número. Push en pila de números.
Pila números: [2.5 3]

Pila operadores: [+]
Expresión: $^2/3$

Token actual: ^
Es operador.
Push en la pila de operadores.
Pila números: [2.5 3]
Pila operadores: [+ ^]
Expresión: $2/3$

Token actual: 2
Es número. Push en pila de números.
Pila números: [2.5 3 2]
Pila operadores: [+ ^]
Expresión: $/3$

Token actual: /
Es operador.
Operador ^ tiene precedencia mayor o igual que /.
Procesando operación $3^2=9$

Push en la pila de operadores.
Pila números: [2.5 9]
Pila operadores: [+ /]
Expresión: 3

Token actual: 3
Es número. Push en pila de números.

Fin de la expresión, procesando lo que queda en las pilas.
Pila números: [2.5 9 3]
Pila operadores: [+ /]
Expresión:

Procesando operación $9/3=3$

Pila números: [2.5 3]
Pila operadores: [+]
Expresión:

Procesando operación $2.5+3=5.5$

Pila números: [5.5]
Pila operadores: []
Expresión:

El resultado de la evaluación es: 5.5

Ejemplo 3 – Expresión con operadores de diferente precedencia y con paréntesis

Escriba la expresión a analizar: $(8 - 1 + 3) * 6 - ((3 + 7) * 2)$

Pila números: []
Pila operadores: []
Expresión: $(8-1+3)*6-((3+7)*2)$

Token actual: (
Es (. Push en la pila de operadores.
Pila números: []
Pila operadores: [(]
Expresión: $8-1+3)*6-((3+7)*2)$

Token actual: 8
Es número. Push en pila de números.
Pila números: [8]
Pila operadores: [(]
Expresión: $-1+3)*6-((3+7)*2)$

Token actual: -
Es operador.
Push en la pila de operadores.
Pila números: [8]
Pila operadores: [(-]
Expresión: $1+3)*6-((3+7)*2)$

Token actual: 1
Es número. Push en pila de números.
Pila números: [8 1]
Pila operadores: [(-]
Expresión: $+3)*6-((3+7)*2)$

Token actual: +
Es operador.
Operador - tiene precedencia mayor o igual que +.
Procesando operación $8-1=7$

Push en la pila de operadores.
Pila números: [7]
Pila operadores: [(+]
Expresión: $3)*6-((3+7)*2)$

Token actual: 3
Es número. Push en pila de números.
Pila números: [7 3]
Pila operadores: [(+]
Expresión: $) * 6 - ((3 + 7) * 2)$

Token actual:)
Es).
Tope de la pila no es (.
Procesando operación $7+3=10$

Pop en la pila de operadores: (
Pila números: [10]

Pila operadores: []
Expresión: $*6-((3+7)*2)$

Token actual: *
Es operador.
Push en la pila de operadores.
Pila números: [10]
Pila operadores: [*]
Expresión: $6-((3+7)*2)$

Token actual: 6
Es número. Push en pila de números.
Pila números: [10 6]
Pila operadores: [*]
Expresión: $-((3+7)*2)$

Token actual: -
Es operador.
Operador * tiene precedencia mayor o igual que -.
Procesando operación $10*6=60$

Push en la pila de operadores.
Pila números: [60]
Pila operadores: [-]
Expresión: $((3+7)*2)$

Token actual: (
Es (. Push en la pila de operadores.
Pila números: [60]
Pila operadores: [- (]
Expresión: $(3+7)*2)$

Token actual: (
Es (. Push en la pila de operadores.
Pila números: [60]
Pila operadores: [- ((]
Expresión: $3+7)*2)$

Token actual: 3
Es número. Push en pila de números.
Pila números: [60 3]
Pila operadores: [- ((]
Expresión: $+7)*2)$

Token actual: +
Es operador.
Push en la pila de operadores.
Pila números: [60 3]
Pila operadores: [- ((+]
Expresión: $7)*2)$

Token actual: 7
Es número. Push en pila de números.
Pila números: [60 3 7]
Pila operadores: [- ((+]

Expresión:)*2)

Token actual:)

Es).

Tope de la pila no es (.

Procesando operación $3+7=10$

Pop en la pila de operadores: (

Pila números: [60 10]

Pila operadores: [- (]

Expresión: *2)

Token actual: *

Es operador.

Push en la pila de operadores.

Pila números: [60 10]

Pila operadores: [- (*]

Expresión: 2)

Token actual: 2

Es número. Push en pila de números.

Pila números: [60 10 2]

Pila operadores: [- (*]

Expresión:)

Token actual:)

Es).

Tope de la pila no es (.

Procesando operación $10*2=20$

Pop en la pila de operadores: (

Fin de la expresión, procesando lo que queda en las pilas.

Pila números: [60 20]

Pila operadores: [-]

Expresión:

Procesando operación $60-20=40$

Pila números: [40]

Pila operadores: []

Expresión:

El resultado de la evaluación es: 40

Ejemplo 4 – Expresión no válida

Escriba la expresión a analizar: 4 + 3 *

Pila números: []
Pila operadores: []
Expresión: 4+3*

Token actual: 4

Es número. Push en pila de números.

Pila números: [4]
Pila operadores: []
Expresión: +3*

Token actual: +

Es operador.

Push en la pila de operadores.

Pila números: [4]
Pila operadores: [+]
Expresión: 3*

Token actual: 3

Es número. Push en pila de números.

Pila números: [4 3]
Pila operadores: [+]
Expresión: *

Token actual: *

Es operador.

Push en la pila de operadores.

Fin de la expresión, procesando lo que queda en las pilas.

Pila números: [4 3]
Pila operadores: [+ *]
Expresión:

Procesando operación 4*3=12

Pila números: [12]
Pila operadores: [+]
Expresión:

Hay un error en la expresión.

Recomendaciones

1. Dividir el problema en subproblemas y resolver cada parte con subrutinas.
2. Eliminar los espacios en blanco de la expresión desde un principio.
3. Hacer una subrutina que obtenga el siguiente token en la expresión.
4. Hacer subrutinas para saber si un token es número, operador o paréntesis.
5. Hacer una subrutina que retorne la precedencia de un operador
6. Hacer una subrutina que se encargue de llevar a cabo los pasos para procesar una operación.
7. Utilizar try/catch para atrapar cualquier error que suceda durante la ejecución.
8. Si desea enviar una pila por parámetro a una subrutina, envíela por referencia (&).
9. Recuerde consultar al profesor o asistente si se encuentra con algún inconveniente.